DBMS MINI PROJECT | 3NF NORMALIZED | MYSQL 26.7 INNODB

Hospital Management System

A Production-Grade Relational Database Architecture with Stored Procedures, Deterministic Functions, Triggers, and ACID Compliance

Database Name: `hospital_db` • **Storage Engine:** InnoDB

Presentation & Viva Voce Demonstration

Problem Statement & Objectives

⚠️ Key Operational Challenges

- **Doctor Double Booking:** Overlapping appointment times lead to doctor collisions and patient wait times.
- **Inpatient Bed Drift:** Lack of atomic updates between patient admission and room availability creates phantom vacancies.
- **Data Anomalies:** De-normalized spreadsheets introduce insertion, update, and deletion anomalies.
- **Unaccountable Auditing:** Financial status edits without automated tracking cause billing discrepancies.

🎯 Engineering Objectives

- **Rigorous 3NF Normalization:** Eliminate transitive and partial functional dependencies across all entities.
- **Active Conflict Protection:** Use Stored Procedures and unique constraints to prevent scheduling collisions.
- **Autonomous Auditing:** Employ event-driven triggers to capture before/after financial state changes.
- **ACID Transaction Enforcement:** Guarantee atomic multi-table state updates via InnoDB rollback/commit.

Database Architecture: 6 Core Entities

#	Core Entity	Primary Key	Attributes & Constraints	Relationship
1	**department**	`department_id`	name (UNIQUE), floor (CHECK ≥ 0), hod	1 : M with `doctor`
2	**doctor**	`doctor_id`	name, specialty, phone (UK), salary (> 0), exp	M : 1 with `department`, 1 : M with `appointment`
3	**room**	`room_no`	room_type (ENUM), charges (≥ 0), availability	1 : M with `patient`
4	**patient**	`patient_id`	name, age (0-120), gender, phone (UK), room_no (FK)	M : 1 with `room`, 1 : 1 with `bill`
5	**appointment**	`appointment_id`	patient_id (FK), doctor_id (FK), date, time, status	UQ: (`doctor_id, date, time`)
6	**bill**	`bill_id`	patient_id (FK, UK), amount (≥ 0), status, mode	1 : 1 enforced via UNIQUE FK

Entity-Relationship (ER) Architecture

Relational Cardinalities

- `DEPARTMENT ||--o{ DOCTOR` (1 to Many: A department employs multiple doctors)
- `DOCTOR ||--o{ APPOINTMENT` (1 to Many: A doctor conducts multiple appointments)
- `PATIENT ||--o{ APPOINTMENT` (1 to Many: A patient books multiple consultations)
- `ROOM ||--o{ PATIENT` (1 to Many: A room accommodates assigned patients)
- `PATIENT ||--|| BILL` (1 to 1: Enforced by `UNIQUE patient_id`)

Referential Action Strategies

- `ON DELETE CASCADE`: Deleting a patient cleans up associated appointments and billing records.
- `ON DELETE RESTRICT`: Department cannot be deleted if active doctors are assigned to it.
- `ON DELETE SET NULL`: Patient's room reference becomes NULL upon room decommission.

Normalization Proof: 1NF → 2NF → 3NF

1st Normal Form

Atomic Values:

- No multi-valued attributes or repeating groups.
- Patient contact phones and addresses are scalar.
- Consultations separated into `appointment` relation.

2nd Normal Form

Full Functional Dependency:

- No partial dependencies on candidate keys.
- Single primary keys guarantee full dependency.
- Composite key in `appointment` determines `status` as a complete unit.

3rd Normal Form

Zero Transitive Dependencies:

- No non-key attribute determines another non-key attribute.
- Department details isolated from `doctor`.
- Room pricing isolated from `patient`.

Stored Procedures: Collision & Bed Allocation

1. sp_schedule_appointment

Validates if the requested doctor is already booked for the date and time slot:

```
SELECT COUNT(*) INTO v_conflict_count
FROM appointment
WHERE doctor_id = p_doctor_id
  AND date = p_date
  AND time = p_time
  AND status != 'Cancelled';

IF v_conflict_count > 0 THEN
    SET p_status_msg = 'ERROR: Doctor is booked...';
ELSE
    INSERT INTO appointment ...
    SET p_status_msg = 'SUCCESS: Scheduled.';
END IF;
```

2. sp_assign_room

Performs atomic bed allocation with state validation:

```
SELECT availability INTO v_room_status
FROM room WHERE room_no = p_room_no;

IF v_room_status != 'Available' THEN
    SET p_result = 'ERROR: Room not available.';
ELSE
    UPDATE patient SET room_no = p_room_no
    WHERE patient_id = p_patient_id;

    UPDATE room SET availability = 'Occupied'
    WHERE room_no = p_room_no;

    SET p_result = 'SUCCESS: Room assigned.';
END IF;
```

Database Views & Stored Functions

Relational Views

1. v_appointment_overview:
Joins 4 tables (`appointment`, `patient`, `doctor`, `department`) to present clinical consultation schedules without exposing doctor payroll data.

2. v_patient_billing_summary:
Executes a `LEFT JOIN` between patients, room rates, and invoices to compute outstanding liabilities.

Stored Function: Department Revenue

```
CREATE FUNCTION fn_get_department_total_revenue(
    p_dept_id INT
) RETURNS DECIMAL(10,2) DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(10,2);
    SELECT IFNULL(SUM(b.amount), 0.00) INTO v_total
    FROM bill b
    JOIN patient p ON b.patient_id = p.patient_id
    JOIN appointment a ON p.patient_id = a.patient_id
    JOIN doctor d ON a.doctor_id = d.doctor_id
    WHERE d.department_id = p_dept_id
      AND b.payment_status = 'Paid';
    RETURN v_total;
END;
```

Autonomous Triggers & Audit Logging

Trigger Implementation

```
CREATE TRIGGER trg_after_bill_payment_update
AFTER UPDATE ON bill
FOR EACH ROW
BEGIN
    IF OLD.payment_status <> NEW.payment_status THEN
        INSERT INTO audit_log (
            table_name, action_type, record_id, details
        )
        VALUES (
            'bill',
            'UPDATE',
            NEW.bill_id,
            CONCAT('Payment status changed from ',
                   OLD.payment_status, ' to ',
                   NEW.payment_status,
                   ' (Amount: ₹', NEW.amount, ')')
        );
    END IF;
END;
```

Trigger Verification Results

Executing payment settlement on Bill #5:

```
UPDATE bill SET payment_status = 'Paid'
WHERE bill_id = 5;
```

Automated Audit Record Created:

Table	Action	Record	Details
bill	UPDATE	5	Changed Pending → Paid

ACID Transactions: Rollback & Commit

1. Rollback Scenario (Undo Changes)

```
START TRANSACTION;
-- Simulate erroneous bed maintenance update
UPDATE room SET availability = 'Maintenance'
WHERE room_no = 302;

-- Revert back to original state
ROLLBACK;

-- Verified: Room 302 reverts to 'Available'
SELECT room_no, availability FROM room
WHERE room_no = 302;
```

2. Commit Scenario (Atomic State Change)

```
START TRANSACTION;
-- Atomic multi-table write
INSERT INTO patient (name, age, gender, phone_no,
                     address, room_no)
VALUES ('Karan Singhania', 35, 'Male',
        '9765012399', 'Pune', 302);

UPDATE room SET availability = 'Occupied'
WHERE room_no = 302;

COMMIT;
-- Both patient record and bed status persist permanently
```

Conclusion & Key Highlights

SUMMARY

1. Relational Rigor

100% 3NF normalized schema with Foreign Key cascading rules, CHECK constraints, and B-Tree indexes for fast search paths.

2. Advanced DBMS Features

Demonstrated Views, Stored Procedures with collision logic, Deterministic Functions, and Triggers for security auditing.

3. Full Working Stack

Integrated with real MySQL 26.7 Server, full verification test suites, and interactive live Web Application on `http://localhost:8080`.